1-Scenario Writing

1.General Scenario

This database is designed for a Hotel or Accommodation Reservation System. It stores and manages data related to guests,room types,individual rooms,reservations and payments.Guests can browse different types of rooms(e.g. "Standard" or "Deluxe Suite") and check their availability for specific dates.A guest must be registered in the system to make a reservation.Each guest has personal and identification details including a National ID(T.C. Kimlik No) which is unique for legal check-in compliance. Each reservation links one guest to one specific physical room for a set period(check-in to check-out date) of time.

Rooms are organized by room types which define capacity and price per night.Each room is linked to one room type and has attributes such as room number,floor and current status(like available,occupied or under maintenance).The system also tracks all payments.A reservation has a total cost and a guest can make one or more payments(as an example a deposit and then the final balance) for a single reservation.This database helps keep all guest and booking data organized,accurate and consistent.
It supports important operations such as:

- **Adding new guests.**
- **Checking room availability for a given date range.**
- **Creating,updating and canceling reservations.**
- **Recording guests` payments.**
- **Automatically updating a room's status(e.g. to "Occupied") when a guest checks in.**
- **Generating reports on revenue,occupancy and guest history.**
- **The database will be used by hotel staffs(receptionists,managers) to manage daily operations and by Guests(via a website/mobile app) to make bookings.**

The main purpose of the database is to:

- **Manage guest identification and contact information.**
- **Manage the hotel's room inventory including room types,pricing and real-time status.**
- **Record all guest reservations with their specific dates and total cost.**
- **Track all payments made toward reservations.**
- **Automatically manage room availability to prevent double-booking.**
- **Provide summarized reports on guests` transactions.**

2. Main Entities and Descriptions

- Guests: Represents clients who make reservations and stay at the hotel.
- RoomTypes: Stores the categories of rooms(e.g. "Standard" or "Delux Suite") including their base price and capacity.
- Rooms: Represents the physical hotel rooms,their number and current operational status (e.g. "Available","Occupied" or “Under Maintance”).
- Reservations: Connects guests to rooms for specific dates and stores reservation details.
- Payments: Records all payment transactions made by guests for their reservations.

3. Functional Specification

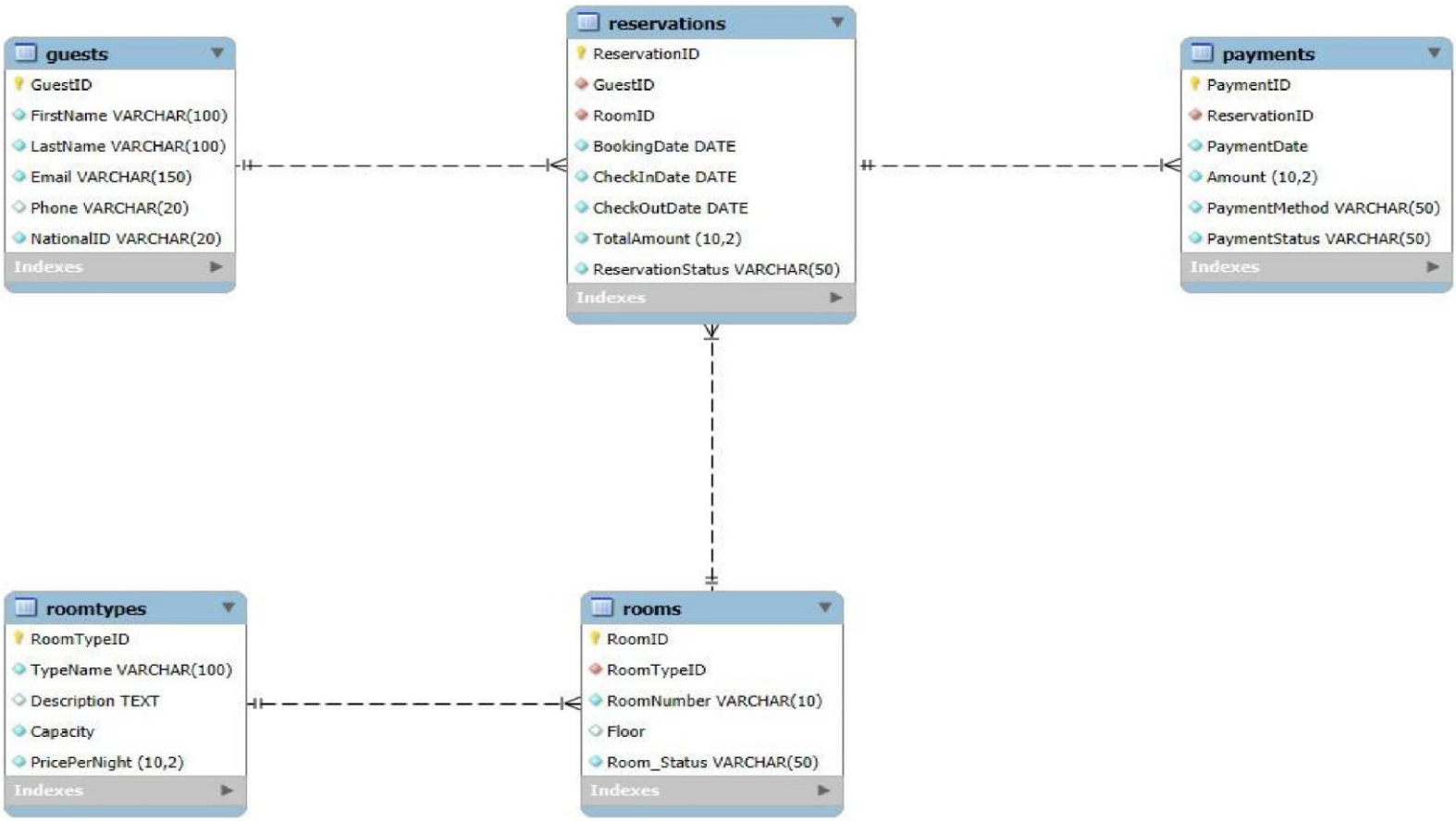
Includes functions for adding guests,creating and managing room types and rooms, recording reservations and processing payments. The system also supports calculating total stay costs,monitoring room availability, generating reports of active or completed reservations,listing all guests and their booking histories and also updating reservation and room statuses automatically.

4. Constraints

- A Guest record must exist before Reservation can be made for that guest.
- A Room and RoomType must exist before they can be booked.
- CheckOutDate must always be later than CheckInDate.
- NationalID and Email address in Guests must be unique.
- A specific Room cannot be booked by two different Reservations if their date ranges overlap(prevention of double-booking).This will be enforced by a Trigger or Stored Procedure.
- Each Room can have only one active reservation at any given time.
- TotalAmount in Reservations must be greater than zero(representing the calculated total cost of the booking at the time of reservation even if payment occurs later).
- A Reservation with associated payments cannot be deleted which ensures financial records are protected(On Delete Restrict).
- Each payment record’s date must not be earlier than the related

- Reservation's BookingDate(since a guest cannot make a payment before the reservation itself has been created).
- ReservationStatus must be one of the valid options (Confirmed,Checked-In, Canceled,Completed).[A Reservation cannot be made for a Room if its Status is "In Cleaning" or "Out of Order"].

2-ER Diagram



3-Normalization

The database schema has been designed and normalized up to the Third Normal Form (3NF) to ensure data integrity and minimize redundancy:

- First Normal Form (1NF): All tables contain atomic values(e.g. Guest names are split into FirstName and LastName) and there are no repeating groups or arrays stored in a single cell.
- Second Normal Form (2NF): All tables have a valid Primary Key and all non-key attributes are fully functional dependent on that primary key.For example,in the Reservations table attributes like CheckInDate depend entirely on the unique ReservationID.
- Third Normal Form (3NF): We eliminated transitive dependencies.A key example is the separation of Room Types.Attributes like PricePerNight and Capacity depend on the *category* of the room,not the specific physical room.By creating a separate RoomTypes table and linking it to the Rooms table via the RoomTypeID foreign key,we adhere to 3NF. This ensures that if a price changes,we update it in one place(RoomTypes) rather than updating every single room row.

4-Tables And Data Entries

```
Create Database Hotel_Reservation_System;

Create Table Guests(
GuestID int auto_increment primary key,
FirstName varchar(100) not null,
LastName varchar(100) not null,
Email varchar(150) not null UNIQUE,
Phone varchar(20) null,
NationalID varchar(20) not null UNIQUE
);



Create Table RoomTypes(
RoomTypeID int auto_increment primary key,
TypeName varchar(100) not null,
Description text,
Capacity int not null,
PricePerNight decimal(10,2) not null
);

Create Table Rooms(
RoomID int auto_increment primary key,
RoomTypeID int not null,
RoomNumber varchar(10) not null UNIQUE,
Floor int null,
Room_Status varchar(50) not null, foreign
key(RoomTypeID) references RoomTypes(RoomTypeID) );

Create Table Reservations(
ReservationID int auto_increment primary key,
GuestID INT not null,
RoomID INT not null,
BookingDate date not null,
CheckInDate date not null,
CheckOutDate date not null,
TotalAmount DECIMAL(10,2) not null,
ReservationStatus varchar(50) not null, foreign key(GuestID)
references Guests(GuestID), foreign key(RoomID) references
Rooms(RoomID),
CONSTRAINT CHK_DateOrder CHECK (CheckOutDate > CheckInDate),
CONSTRAINT CHK_BookingLogic CHECK (CheckInDate >= BookingDate),
CONSTRAINT CHK_TotalAmount CHECK (TotalAmount > 0)  );

Create Table Payments(
PaymentID int auto_increment primary key,
ReservationID int not null,
PaymentDate datetime not null,
Amount decimal(10,2) not null,
PaymentMethod varchar(50) not null,
PaymentStatus varchar(50) not null,
foreign key(ReservationID) references Reservations(ReservationID) ON
DELETE restrict
);

Insert into Guests(FirstName,LastName,Email,Phone,NationalID) values
('Kerem','YILMAZ','Kerem.Y@gmail.com','532-0101','11111111110'),
('Ayşe','YILDIRIM','Ayşe.Y@gmail.com','532-0102','22222222220'),
('Emre','ÖZKAN','Emre.Ö@gmail.com','532-0103','33333333330'),
('Michael','BROWN','Michael.B@gmail.com','532-0104','44444444440'),
('Hamit','Şahin','Hamit.Ş@gmail.com','532-0105','55555555550'),
```

```
('Zeynep','Öztürk','Zeynep.Ö@gmail.com','532-0106','66666666660'),
('Mustafa','ARSLAN','Mustafa.A@gmail.com','532-0107','77777777770'),
('John','SMITH','John.S@gmail.com','532-0108','88888888880'),
('Hüseyin','YILDIZ','Hüseyin.Y@gmail.com','532-0109','99999999990'),
('William','JONES','William.J@gmail.com','532-0110','10101010100');

Insert into RoomTypes(TypeName,Description,Capacity,PricePerNight) values
('Economy Single','Small,budget-friendly room without a view',1,80.00),
('Standard Single','Usual room for solo travelers',1,100.00),
('Standard Double','Comfortable room with two beds',2,200.00),
('Deluxe Suite','Luxury suite with city view and private terrace',2,500.00),
('Family Room','Spacious room for families with 4 beds',4,350.00),
('Presidential Suite','A presidential suite will consist of multiple rooms with opulent furnishings and
amenities',30,1000.00),
('Bridal Suite','A suite of rooms in a hotel for the use of a newly married couple',2,500.00),
('Honeymoon Suite','A suite with special amenities primarily aimed at couples and newlyweds',2,650.00),
('King Suite','This is a type of hotel room that contains a larger double bed,namely the king size',8,800.00),
('Junior Suite','A spacious room with a small sitting area and king bed',2,280.00);

Insert into Rooms(RoomTypeID,RoomNumber,Floor,Room_Status) values
(1,'101',1,'Available'),
(2,'102',1,'Available'),
(3,'103',1,'Under Maintenance'),
(4,'201',2,'Available'),
(5,'202',2,'Available'),
(6,'203',2,'Occupied'),
(7,'301',3,'Available'),
(8,'302',3,'Occupied'),
(9,'303',3,'Available'),
(10,'401',4,'Available');

Insert into
Reservations(GuestID,RoomID,BookingDate,CheckInDate,CheckOutDate,TotalAmount,ReservationStatus) values
(1,6,'2025-02-01','2025-02-20','2025-02-25',5000.00,'Confirmed'),
(2,8,'2025-11-02','2025-11-21','2025-11-23',1300.00,'Confirmed'),
(3,1,'2025-08-25','2025-09-05','2025-09-10',400.00,'Completed'),
(4,10,'2025-11-05','2025-12-01','2025-12-05',1120.00,'Confirmed'),
(5,2,'2025-03-10','2025-0315','2025-03-16',100.00,'Canceled'),
(6,3,'2024-12-12','2025-01-10','2025-01-15',1000.00,'Confirmed'),
(7,4,'2025-04-14','2025-04-25','2025-04-30',2500.00,'Confirmed'),
(8,7,'2025-11-15','2025-12-20','2025-12-25',2500.00,'Confirmed'),
(9,9,'2025-09-16','2025-09-20','2025-09-22',1600.00,'Completed'),
(10,5,'2025-11-18','2025-12-30','2026-01-02',1050.00,'Confirmed');

Insert into Payments(ReservationID,PaymentDate,Amount,PaymentMethod,PaymentStatus) values
(1,'2025-02-01 10:00:00',1000.00,'Credit Card','Completed'),
(1,'2025-02-25 12:00:00',4000.00,'Cash','Pending'),
(2,'2025-11-02 14:30:00',1300.00,'Credit Card','Completed'),
(3,'2025-08-25 09:00:00',400.00,'Bank Transfer','Completed'),
(4,'2025-11-05 11:00:00',500.00,'Credit Card','Completed'),
(5,'2025-03-10 08:00:00',100.00,'Credit Card','Refunded'),
(6,'2024-12-12 15:45:00',1000.00,'Credit Card','Completed'),
(7,'2025-04-14 16:20:00',2500.00,'Cash','Completed'),
(8,'2025-11-15 13:10:00',2500.00,'Bank Transfer','Completed'),
(9,'2025-09-22 10:00:00',1600.00,'Cash','Completed'),
(10,'2025-11-18 09:30:00',1050.00,'Credit Card','Completed');
```

5-SQL Queries

1. Stored Procedures

Structure 1: sp_GetAvailableRooms Code to
Create Structure:

```
DELIMITER //
CREATE PROCEDURE sp_GetAvailableRooms(IN p_CheckInDate DATE, IN p_CheckOutDate DATE) BEGIN
   SELECT r.RoomNumber, rt.TypeName, rt.PricePerNight, r.Room_Status     FROM
Rooms r
   JOIN RoomTypes rt ON r.RoomTypeID = rt.RoomTypeID
   WHERE r.Room_Status != 'Under Maintenance'
   AND r.RoomID NOT IN (
      SELECT res.RoomID FROM Reservations res
      WHERE res.ReservationStatus IN ('Confirmed', 'Checked-In')
      AND ((p_CheckInDate BETWEEN res.CheckInDate AND res.CheckOutDate)
      OR (p_CheckOutDate BETWEEN res.CheckInDate AND res.CheckOutDate)        OR
(res.CheckInDate BETWEEN p_CheckInDate AND p_CheckOutDate))
   );
END //
DELIMITER ;
```

Explanation:I used this procedure to encapsulate the complex logic required to check room availability.It uses a JOIN to get room type details and a NOT IN subquery to exclude rooms that have conflicting reservations during the requested dates.In real world example this simplifies the booking process for the front desk staff.

Query Question:Find all rooms that are available for a stay between January 1,2026 and January 5,2026.

Query Code:CALL sp_GetAvailableRooms('2026-01-01', '2026-01-05');

Query Output:

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

RoomNumber	TypeName	PricePerNight	Room_Status
101	Economy Single	80.00	Available
102	Standard Single	100.00	Available
201	Deluxe Suite	500.00	Available
202	Family Room	350.00	Available
203	Presidential Suite	1000.00	Occupied
301	Bridal Suite	500.00	Available
302	Honeymoon Suite	650.00	Occupied
303	King Suite	800.00	Available

Result 1 | Result Grid | Form Editor | Read Only

Structure 2: sp_GuestHistory Code to
Create Structure:

```
DELIMITER //
CREATE PROCEDURE sp_GuestHistory(IN p_GuestID INT)
BEGIN
   SELECT g.FirstName, g.LastName, r.CheckInDate, r.CheckOutDate, rm.RoomNumber, r.TotalAmount,
r.ReservationStatus
   FROM Reservations r
   JOIN Guests g ON r.GuestID = g.GuestID
   JOIN Rooms rm ON r.RoomID = rm.RoomID
   WHERE r.GuestID = p_GuestID
   ORDER BY r.BookingDate DESC;
END //
DELIMITER ;
```

Explanation:This procedure creates a report of a specific guest's interaction with the hotel. It uses multiple JOIN statements to link the Reservation table with Guests and Rooms and ORDER BY to show the most recent history first.

Query Question:Show the complete booking history for the guest with ID 1 (Kerem Yılmaz).

Query Code:

```
CALL sp_GuestHistory(1);
```

Query Output:

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

FirstName	LastName	CheckInDate	CheckOutDate	RoomNumber	TotalAmount	ReservationStatus
Kerem	YILMAZ	2026-03-01	2026-03-05	101	400.00	Confirmed
Kerem	YILMAZ	2026-03-01	2026-03-05	101	400.00	Canceled
Kerem	YILMAZ	2025-02-20	2025-02-25	203	5000.00	Confirmed

Result Grid | Form Editor

Result 1 | Read Only

2. Stored Functions

Structure 3: fn_CalculateYearlyRevenue

Code to Create Structure:

```
DELIMITER //
CREATE FUNCTION fn_CalculateYearlyRevenue(p_Year INT) RETURNS DECIMAL(10, 2) DETERMINISTIC
BEGIN
   DECLARE v_Total DECIMAL(10, 2);
   SELECT SUM(Amount) INTO v_Total FROM Payments
   WHERE YEAR(PaymentDate) = p_Year AND PaymentStatus = 'Completed';     RETURN
IFNULL(v_Total, 0.00);
END //
DELIMITER ;
```

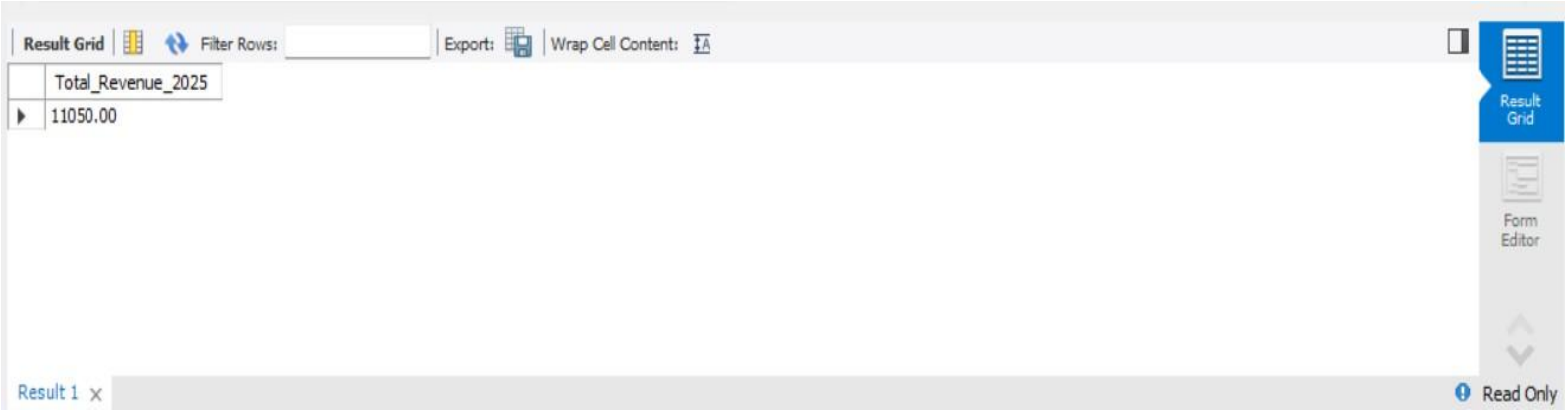
Explanation:This function calculates the total income for a specific year using the aggregate function SUM().It ensures financial accuracy by only counting 'Completed' payments and filtering by year.

Query Question:What was the total confirmed revenue for the year 2025?

Query Code:

```
SELECT fn_CalculateYearlyRevenue(2025) AS Total_Revenue_2025;
```

Query Output:



Structure 4: fn_CountAvailableRooms Code to Create Structure:

```
DELIMITER //
CREATE FUNCTION fn_CountAvailableRooms(p_RoomTypeID INT) RETURNS INT DETERMINISTIC
BEGIN
   DECLARE v_Count INT;
   SELECT COUNT(*) INTO v_Count FROM Rooms
   WHERE RoomTypeID = p_RoomTypeID AND Room_Status = 'Available';     RETURN
v_Count;
END //
DELIMITER ;
```

Explanation:This function uses COUNT(*) to return the exact number of rooms currently available for a specific category.This is useful for quick inventory checks.

Query Question:How many 'Standard Single' rooms(Type ID 2) are currently marked as available?

Query Code:

```
SELECT fn_CountAvailableRooms(2) AS Available_Standard_Singles;
```

Query Output:

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Available_Singles
1

Result 1 | Read Only

3. Views

Structure 5: vw_DetailedReservations Code to Create Structure:

```
CREATE OR REPLACE VIEW vw_DetailedReservations AS
SELECT r.ReservationID, CONCAT(g.FirstName, ' ', g.LastName) AS GuestName,
    rm.RoomNumber, rt.TypeName, r.CheckInDate, r.CheckOutDate, r.ReservationStatus FROM
Reservations r
JOIN Guests g ON r.GuestID = g.GuestID
JOIN Rooms rm ON r.RoomID = rm.RoomID
JOIN RoomTypes rt ON rm.RoomTypeID = rt.RoomTypeID;
```

Explanation:This view simplifies complex data retrieval by joining four different tables(Reservations, Guests,Rooms, RoomTypes) into a single virtual table.It allows staff to see all reservation details without writing complex joins repeatedly.

Query Question:List all details for current reservations from the view.

Query Code:

```
SELECT * FROM vw_DetailedReservations;
```

Query Output:

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

ReservationID	GuestName	RoomNumber	TypeName	CheckInDate	CheckOutDate	ReservationStatus
3	Emre ÖZKAN	101	Economy Single	2025-09-05	2025-09-10	Completed
11	Kerem YILMAZ	101	Economy Single	2026-03-01	2026-03-05	Canceled
12	Kerem YILMAZ	101	Economy Single	2026-03-01	2026-03-05	Confirmed
5	Hamit Şahin	102	Standard Single	2025-03-15	2025-03-16	Canceled
6	Zeynep Öztürk	103	Standard Double	2025-01-10	2025-01-15	Confirmed
7	Mustafa ARSLAN	201	Deluxe Suite	2025-04-25	2025-04-30	Confirmed
10	William JONES	202	Family Room	2025-12-30	2026-01-02	Confirmed
1	Kerem YILMAZ	203	Presidential Suite	2025-02-20	2025-02-25	Confirmed
8	John SMITH	301	Bridal Suite	2025-12-20	2025-12-25	Confirmed

Reservations 1 | Read Only

Structure 6: vw_RoomTypePerformance Code to Create Structure:

```
CREATE OR REPLACE VIEW vw_RoomTypePerformance AS
SELECT
   rt.TypeName,
   COUNT(r.ReservationID) AS TotalBookings,
   SUM(r.TotalAmount) AS TotalRevenue,
   AVG(r.TotalAmount) AS AvgBookingValue
FROM Reservations r
JOIN Rooms rm ON r.RoomID = rm.RoomID
JOIN RoomTypes rt ON rm.RoomTypeID = rt.RoomTypeID
GROUP BY rt.TypeName
HAVING TotalRevenue > 0;
```

Explanation:This analytical view satisfies the requirements for GROUP BY, HAVING, SUM and AVG. It aggregates reservation data by Room Type to analyze which room categories perform best financially.

Query Question:Show the performance statistics (bookings,revenue,average value) for all room types that have generated revenue.

Query Code:

```
SELECT * FROM vw_RoomTypePerformance;
```

Query Output:

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

TypeName	TotalBookings	TotalRevenue	AvgBookingValue
Economy Single	3	1200.00	400.000000
Standard Single	1	100.00	100.000000
Standard Double	1	1000.00	1000.000000
Deluxe Suite	1	2500.00	2500.000000
Family Room	1	1050.00	1050.000000
Presidential Suite	1	5000.00	5000.000000
Bridal Suite	1	2500.00	2500.000000
Honeymoon Suite	1	1300.00	1300.000000
King Suite	1	1600.00	1600.000000
Junior Suite	2	2240.00	1120.000000

vw_RoomTypePerformance 2 | Result Grid | Form Editor | Field Types | Read Only

4. Triggers

Structure 7: trg_PreventDoubleBooking Code to Create Structure:

```
DELIMITER //
CREATE TRIGGER trg_AutoUpdateRoomStatus
AFTER UPDATE ON Reservations
FOR EACH ROW
BEGIN
   IF NEW.ReservationStatus = 'Checked-In' THEN
      UPDATE Rooms SET Room_Status = 'Occupied' WHERE RoomID = NEW.RoomID;
   ELSEIF NEW.ReservationStatus IN ('Completed', 'Canceled') THEN
      UPDATE Rooms SET Room_Status = 'Available' WHERE RoomID = NEW.RoomID;
   END IF;
END //
DELIMITER ;
```

Explanation:This trigger enforces the business rule that a room cannot be double-booked.It fires BEFORE INSERT and uses a logic check (IF/THEN) to validate dates.If an overlap is found,it raises an error and stops the insert.

Query Question:Attempt to book Room 6 for dates that overlap with an existing confirmed reservation (Feb 1 - Feb 25).

Query Code:

```
-- This query is expected to fail with an error
INSERT INTO Reservations (GuestID, RoomID, BookingDate, CheckInDate, CheckOutDate, TotalAmount, ReservationStatus)
VALUES (2, 6, CURDATE(), '2025-02-10', '2025-02-15', 500.00, 'Confirmed');
```

Query Output:

8	22:09:48	INSERT INTO Reservations(GuestID, RoomID, BookingDate, CheckInDate, CheckOutDate, TotalAmount, ...	Error Code: 3819. Check constraint 'CHK_BookingLogic' is violated.	0.000 sec

Structure 8: trg_AutoUpdateRoomStatus Code to
Create Structure:

```
DELIMITER //
CREATE TRIGGER trg_AutoUpdateRoomStatus
AFTER UPDATE ON Reservations
FOR EACH ROW
BEGIN
   IF NEW.ReservationStatus = 'Checked-In' THEN
      UPDATE Rooms SET Room_Status = 'Occupied' WHERE RoomID = NEW.RoomID;
   ELSEIF NEW.ReservationStatus IN ('Completed', 'Canceled') THEN
      UPDATE Rooms SET Room_Status = 'Available' WHERE RoomID = NEW.RoomID;
   END IF;
END //
DELIMITER ;
```

Explanation:This trigger automates the maintenance of room statuses in the Rooms table.It listens for updates on the Reservations table and automatically changes the physical room status to 'Occupied' or 'Available' based on the reservation status.
Query Question:Check-in a guest(Reservation ID 2) and verify the room status changes automatically.

Query Code:

```
UPDATE Reservations SET ReservationStatus = 'Checked-In' WHERE ReservationID = 2;
SELECT * FROM Rooms WHERE RoomID = 8; -- Checking status of Room 8 (assigned to Res 2)
```

Query Output:

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

RoomID	RoomTypeID	RoomNumber	Floor	Room_Status
8	8	302	3	Available
NULL	NULL	NULL	NULL	NULL

Rooms 14 ×

Result Grid | Form Editor | Apply | Revert

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

RoomID	RoomTypeID	RoomNumber	Floor	Room_Status
8	8	302	3	Occupied
NULL	NULL	NULL	NULL	NULL

Rooms 16 | Apply | Revert

5. Transactions

Structure 9: sp_CreateResWithPay

Code to Create Structure:

```
DELIMITER //
CREATE PROCEDURE sp_CreateResWithPay(
   IN p_GuestID INT, IN p_RoomID INT, IN p_In DATE, IN p_Out DATE, IN p_Total DECIMAL(10,2), IN p_Pay
DECIMAL(10,2)
)
BEGIN
   DECLARE EXIT HANDLER FOR SQLEXCEPTION ROLLBACK;
   START TRANSACTION;
   INSERT INTO Reservations (GuestID, RoomID, BookingDate, CheckInDate, CheckOutDate, TotalAmount,
ReservationStatus)
   VALUES (p_GuestID, p_RoomID, CURDATE(), p_In, p_Out, p_Total, 'Confirmed');

   SET @NewID = LAST_INSERT_ID();

   INSERT INTO Payments (ReservationID, PaymentDate, Amount, PaymentMethod, PaymentStatus)     VALUES
(@NewID, NOW(), p_Pay, 'Credit Card', 'Completed');
   COMMIT;
   SELECT 'Transaction Successful' AS Status, @NewID AS NewReservationID; END //
DELIMITER ;
```

Explanation:This transaction ensures data integrity during the booking process.It wraps the creation of a Reservation and its initial Payment into a single atomic unit.If either insert fails the ROLLBACK command undoes all changes to preventing data inconsistency.

Query Question:Create a new reservation for Guest 1 in Room 1 for March 2026 with an immediate deposit.

Query Code:

```
CALL sp_CreateResWithPay(1, 1, '2026-03-01', '2026-03-05', 400.00, 100.00);
```

Query Output:

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Status	NewReservationID
Transaction Successful	13

Result 3 | Read Only

Structure 10: sp_CancelResWithRefund Code to
Create Structure:

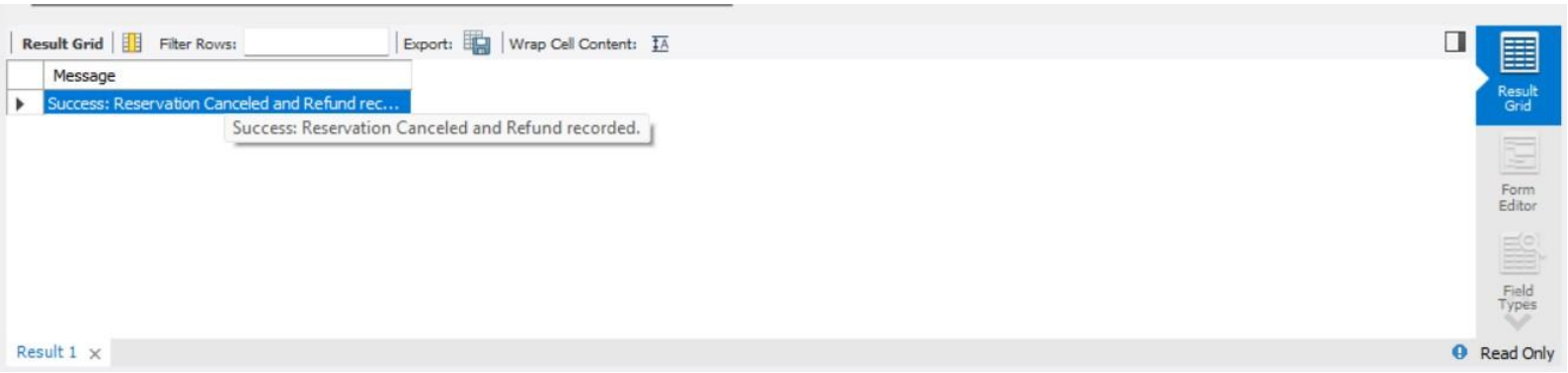
```
DELIMITER //
CREATE PROCEDURE sp_CancelResWithRefund(IN p_ResID INT, IN p_Refund DECIMAL(10,2)) BEGIN
   DECLARE EXIT HANDLER FOR SQLEXCEPTION ROLLBACK;
   START TRANSACTION;
   UPDATE Reservations SET ReservationStatus = 'Canceled' WHERE ReservationID = p_ResID;

   INSERT INTO Payments (ReservationID, PaymentDate, Amount, PaymentMethod, PaymentStatus)
VALUES (p_ResID, NOW(), p_Refund, 'System', 'Refunded');     COMMIT;
   SELECT 'Transaction Successful' AS Status, 'Refund Recorded' AS Action; END //
DELIMITER ;
```

Explanation:This transaction handles the complex logic of cancellation.It ensures that when a reservation status is updated to 'Canceled',a corresponding financial record for the refund is created simultaneously.

Query Question:Cancel the reservation created in the previous step and issue a refund of 100.00.

Query Code:

```
-- Assuming previous ID was 2
CALL sp_CancelResWithRefund(2,100.00);
```

Query Output: